

程序设计实践

The Practice of Programming

可移植性(Portability)

关于移植

什么是移植？

对于一个能够在一个环境下正常工作的程序，试图把它移到另一个环境（处理器，操作系统或者编译系统等）下编译，运行。

什么是可移植性？

移植过程需要修改的东西越少，就说明程序的可移植性越好，反之可移植性越差。



为什么要移植

众多客户的硬件平台，操作系统等系统软件千差万别。

- 后端大型软件的客户需求不同，例如有的偏好IBM系统，有的偏好HP系统。
- 大量APP都要适应各种品牌的PC，PAD，手机等终端。

硬件，操作系统，编译系统等总是在不断升级换代。

- 不断有曾经著名的服务器和操作系统被市场淘汰。
- 服务器巨头之间的并购，也使得某些服务器品牌被取消发展计划。

运营成本的考虑，需要更廉价的硬件或者系统软件。

- 越来越多的小型机系统，被廉价的Linux服务器取代。
- 昂贵的Oracle数据库，替换为免费的mysql

可靠性的考虑，需要更可靠的硬件和操作系统环境。

性能的考虑，需要更高性能的硬件。

希望将程序推广到全球各个角落。



本章内容

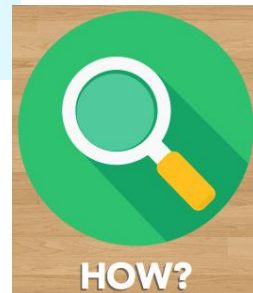
影响移植的因素

- 语言的因素
- 操作系统的因素
- 字节序
- 数据库和SQL
- ...

从系统软件的角度看待可移植性的问题

国际化的相关问题

改善程序可移植性的方法



影响移植的因素

同一个程序设计语言在不同环境下的差异：

- 语言标准化不及时
- 语言的功能扩展
- 编译器实现的差异
- 硬件和操作系统的影响

头文件和库的差异：

- 库的标准比语言更混乱
- 厂商的竞争往往导致标准被故意突破
- 库的不兼容升级

数据交换格式的差异

- 文本控制符的差异
- 字节序的问题
- 字符编码的问题

用户语言和文化差异

- 不同的语言
- 特殊数据的不同格式要求

C语言标准的演化

高版本提供了更强大的功能。

C18 (ISO/IEC 9899:2018)

- C11的修订，没有增加新特性

C11 (ISO/IEC 9899:2011)

- alignof
- fopen的"x"模式
- 各种Unicode字符串类型定义
- ...

C99 (ISO/IEC 9899:1999)

- 双斜线行注释
- inline函数
- 更丰富，更标准化的整数类型定义
- bool类型以及true, false
- 可变长数组
- 灵活的初始化方式
- ...

C90 (ISO/IEC 9899:1990)

- 目前几乎所有编译器都支持的版本
- 也称为K&R C，因为此标准的基础是Kernighan和Ritchie合著的The C Programming Language。
- 也称为ANSI C，因为1989年，ANSI首先对C语言进行了标准化。1990年ANSI C被ISO采纳
- C89, C90是此版本的两个简称

基于低版本写的程序有更好的可移植性。

C++语言标准的演化

C++14, C++17, C++20 ...



C++11 (ISO/IEC 14882:2011)

- 支持auto, decltype
- 支持列表初始化
- 支持std::function, std::bind, lambda表达式
- 基于范围的for循环
- 支持std::regex
- ...

C++98 (ISO/IEC 14882:1998)

- 目前绝大部分编译器支持的C++标准
- 标准模板库STL
- 运行时类识别
- mutable
- bool
- 条件内定义变量
- ...



标准化之前的C++

- 1983年正式命名C++
- 1986年, 引入虚函数, 运算符重载, 引用, const, new/delete, 双斜线行注释等。
- 1990年, 引入命名空间, 异常, 模板, 类嵌套。



Java语言标准的演化

OpenJDK

开始于2006年的OpenJDK



Java
Community
Process

JCP负责接收JSRs(Java Specification Request), 维护Java标准



Microsoft

昙花一现的Visual J++



JDK1.0-1.1

J2SE1.2-1.4

Java SE 5, 6

Java SE 7, 8, 11, 17



ORACLE

SUN时代

ORACLE时代

1996

1999

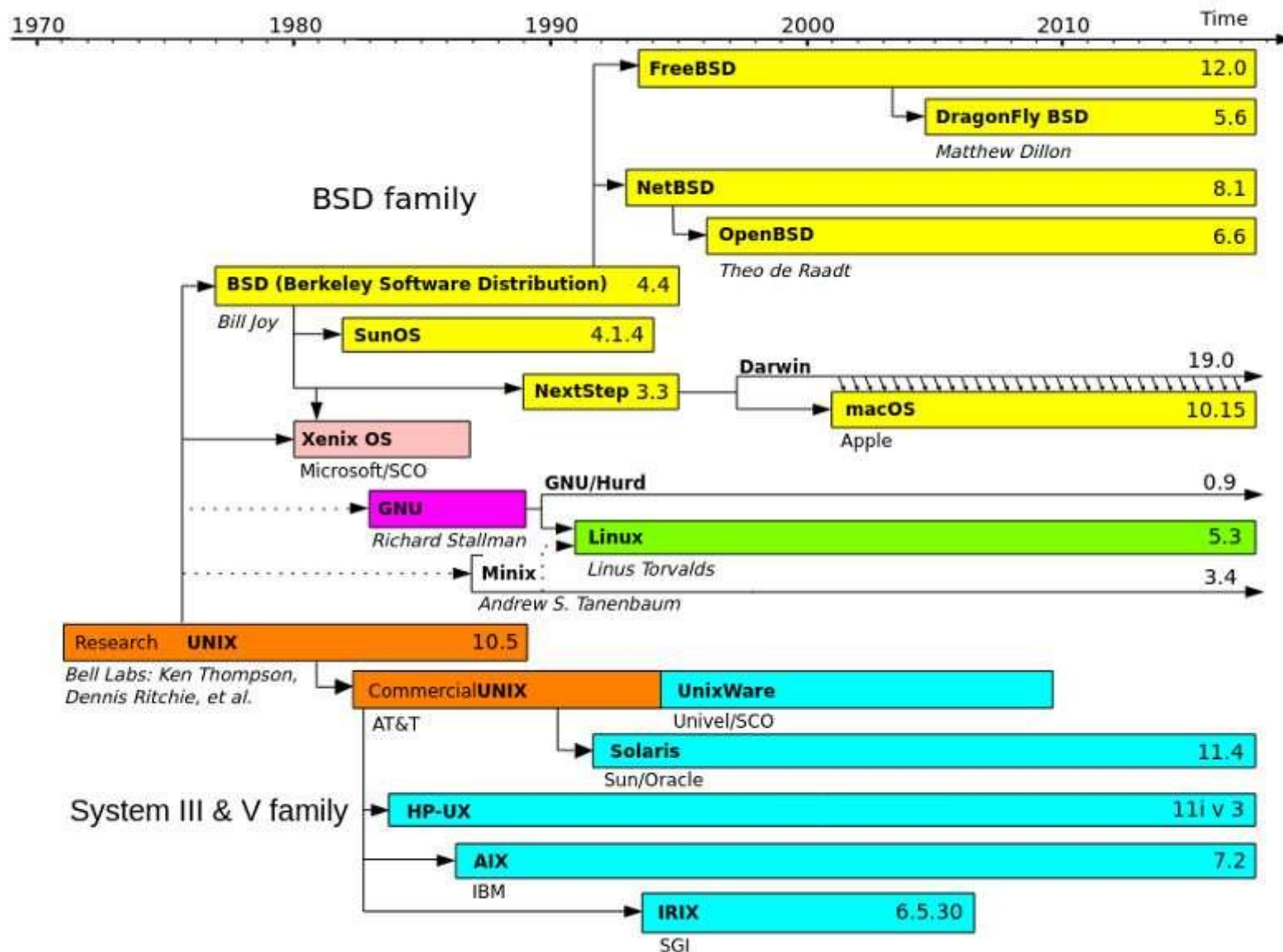
2004

2009

2011

2021

Unix/Linux的版本历史



C/C++ – 数据类型的大小

char: 绝大部分环境下，都是1个字节

short: 绝大部分环境下，都是2个字节

int:

- 大部分环境下，都是4个字节。
- 少数比较古老的环境，嵌入式系统是2字节

long:

- **unix/linux**阵营，32位系统是4字节，64位系统是8字节
- 微软阵营，都是4字节

long long:

- **c99**支持的新类型，肯定是8字节

void*:

- 32位系统是4字节，64位系统是8字节

time_t:

- 32位系统是4字节，64位系统是8字节

C/C++ – char的符号

char到底是有符号的还是无符号的，C和C++中并没有给出明确的规定。

```
char c;  
c = getchar();  
if(c == EOF)  
    printf("end of file");
```

```
int c;  
c = getchar();  
if(c == EOF)  
    printf("end of file");
```

虽然第一种写法在大部分环境下都可以正确执行，但是为了更好的兼容性，应该使用第二种写法。

C/C++ – char的符号

```
void printHex(const char* s, int length)
{
    for(int i = 0; i < length; i++)
        printf("%02X ", s[i]);
}
```

```
void printHex(const char* s, int length)
{
    for(int i = 0; i < length; i++)
        printf("%02X ", (unsigned char)(s[i]));
}
```

C/C++ - 函数参数求值次序

```
char c = 'A';  
printf("%c%c\n", c, ++c);
```



BB



'B'



'B'

```
char c = 'A';  
printf("%c%c\n", c, ++c);
```



AB



'A'

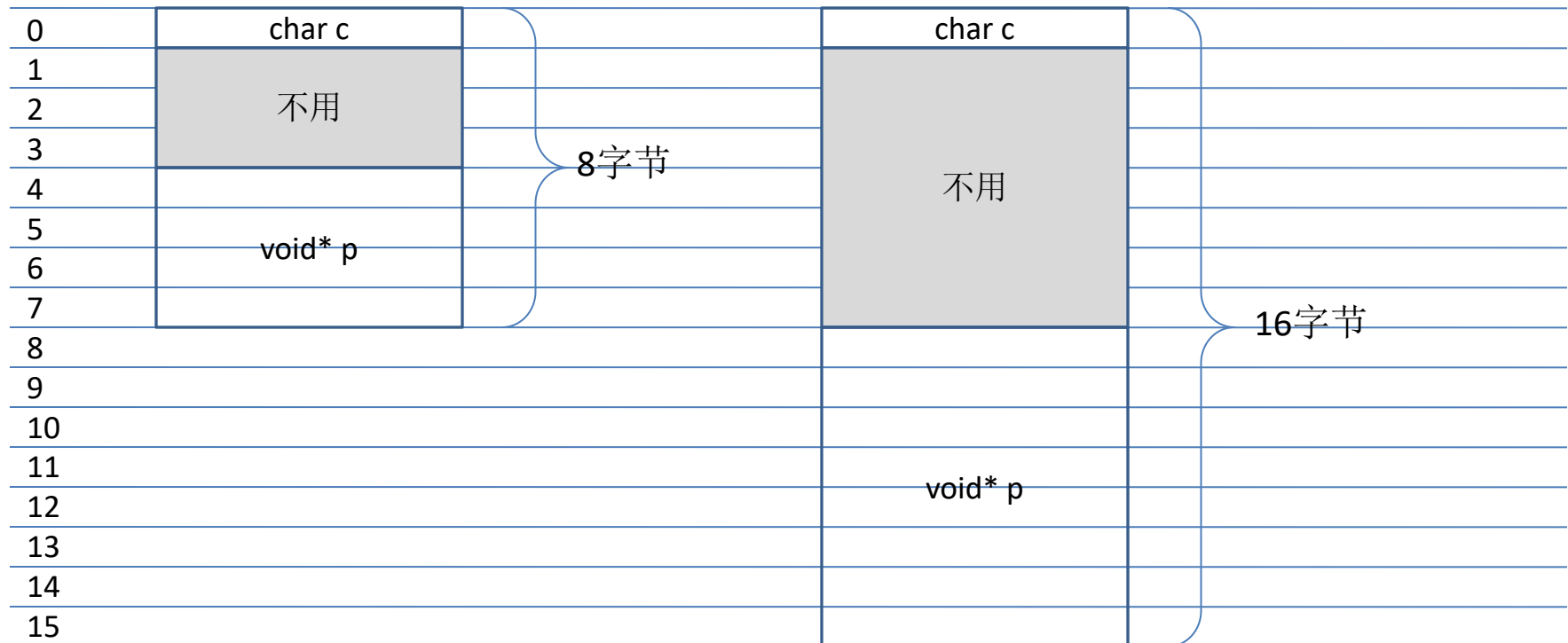


'B'

函数参数的执行次序是不一定的，依赖编译器的实现。有的编译器会从左到右执行，有的编译器会从右到左执行。

C/C++ - 结构中成员的对齐方式

```
struct X{
    char c;
    void* p;
}
printf("%d\n", sizeof(X));
```



C/C++ - 变量的作用域

```
for(int i = 0; i < MAX; i++)
```

```
...
```

```
for(int i = 0; i < MAX; i++)
```

```
...
```

```
for(int i = 0; i < MAX; i++)
```

```
...
```

```
for(i = 0; i < MAX; i++)
```

```
...
```

```
int i;
```

```
for(i = 0; i < MAX; i++)
```

```
...
```

```
for(i = 0; i < MAX; i++)
```

```
...
```

C/C++ - strdup函数

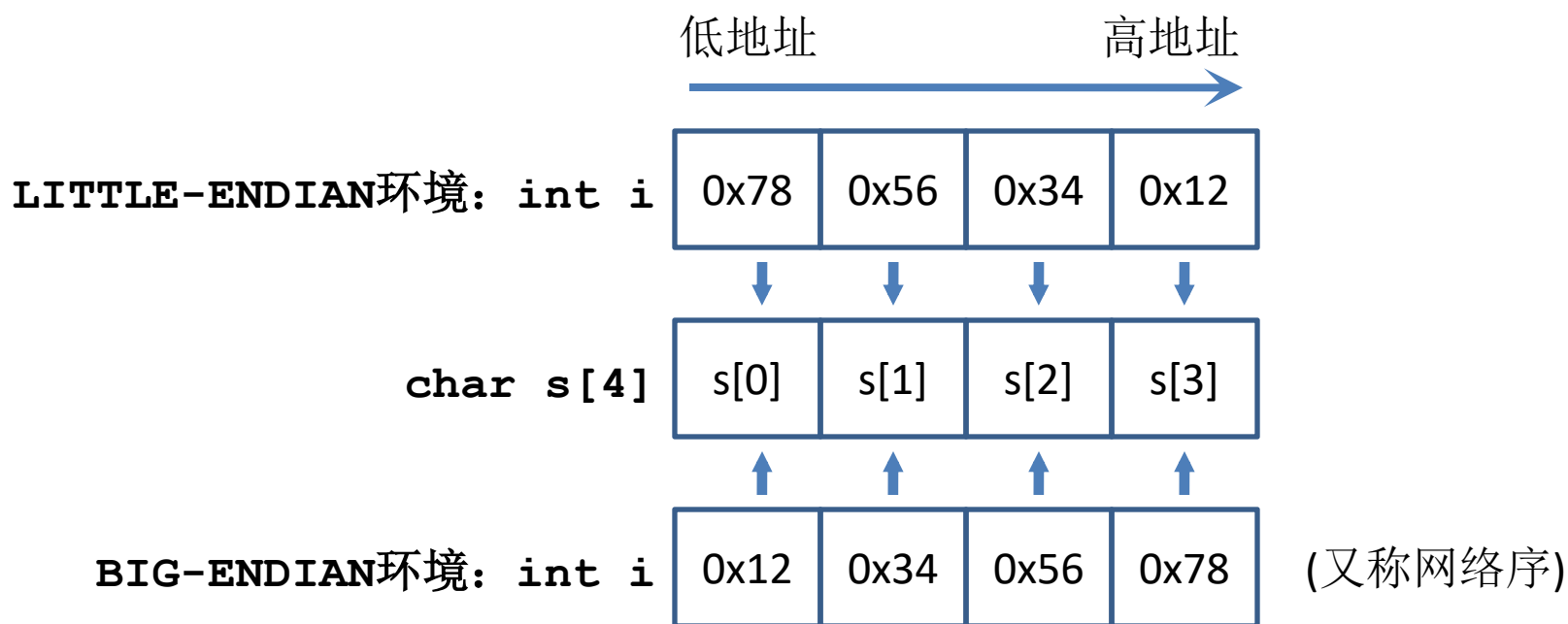
```
#include <string.h>
char *strdup(const char *s) {
    size_t size = strlen(s) + 1;
    char *p = malloc(size);
    if (p) {
        memcpy(p, s, size);
    }
    return p;
}
```

strdup是一个很多程序员都熟悉的函数，它申请一块新的内存复制了字符串并返回。因为这个函数违背了“接口不应偷偷申请内存”这个原则，所以这个函数一直不被C语言的系列标准接纳，有的系统支持，有的则坚决不支持。所以经常会引起一些移植性的问题。

所以，不要使用这个库函数。也不要试图自己写一个类似的函数。

字节序

```
int i = 0x12345678;
unsigned char s[4];
memcpy(s, &i, 4);
printf("%02X %02X %02X %02X\n", s[0], s[1], s[2], s[3]);
```



字节序 - encodeIntBE

```
int encodeIntBE(int i, char* p) // 将整数编码为BIG-ENDIAN格式存入p
{
#ifdef _LITTLE_ENDIAN
    i = (((i) & 0xff000000u) >> 24)
        | (((i) & 0x00ff0000u) >> 8)
        | (((i) & 0x0000ff00u) << 8)
        | (((i) & 0x000000ffu) << 24);
#endif
    memcpy(p, (char*)(&i), sizeof(int));
    return sizeof(int);
}
```

```
#include <arpa/inet.h>
int encodeIntBE(int i, char* p) // 将整数编码为BIG-ENDIAN格式存入p
{
    i = htonl(i);
    memcpy(p, (char*)(&i), sizeof(int));
    return sizeof(int);
}
```

不要为了实现可移植性随意增加宏定义，首先寻找是否有系统支持的可移植性好的函数可用。

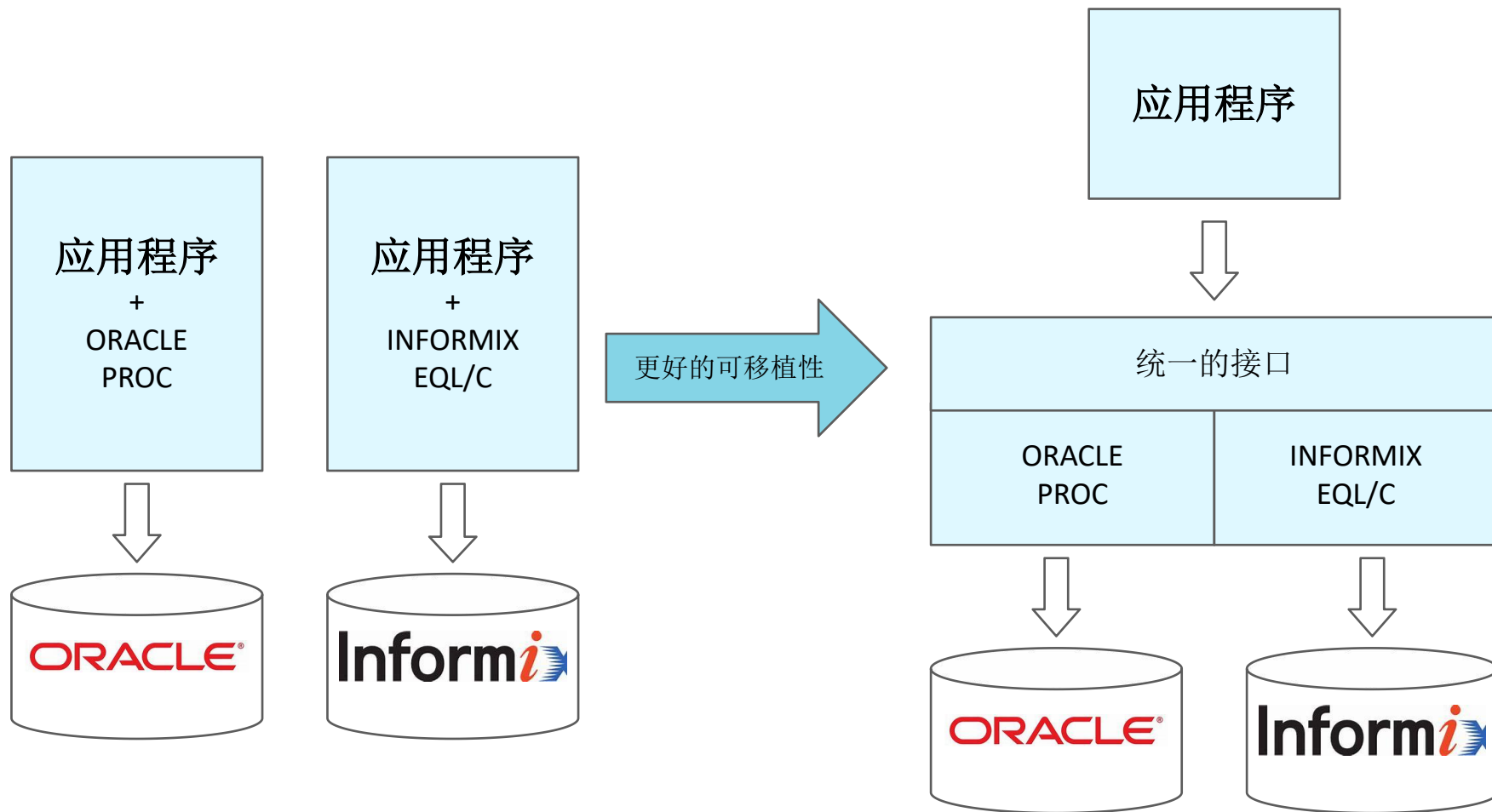
字节序 - encodeIntLE

```
int encodeIntLE(int i, char* p) // 将整数编码为LITTLE-ENDIAN格式存入p
{
#ifdef _BIG_ENDIAN
    i = (((i) & 0xff000000u) >> 24)
        | (((i) & 0x00ff0000u) >> 8)
        | (((i) & 0x0000ff00u) << 8)
        | (((i) & 0x000000ffu) << 24);
#elseif
    memcpy(p, (char*)(&i), sizeof(int));
    return sizeof(int);
}
```

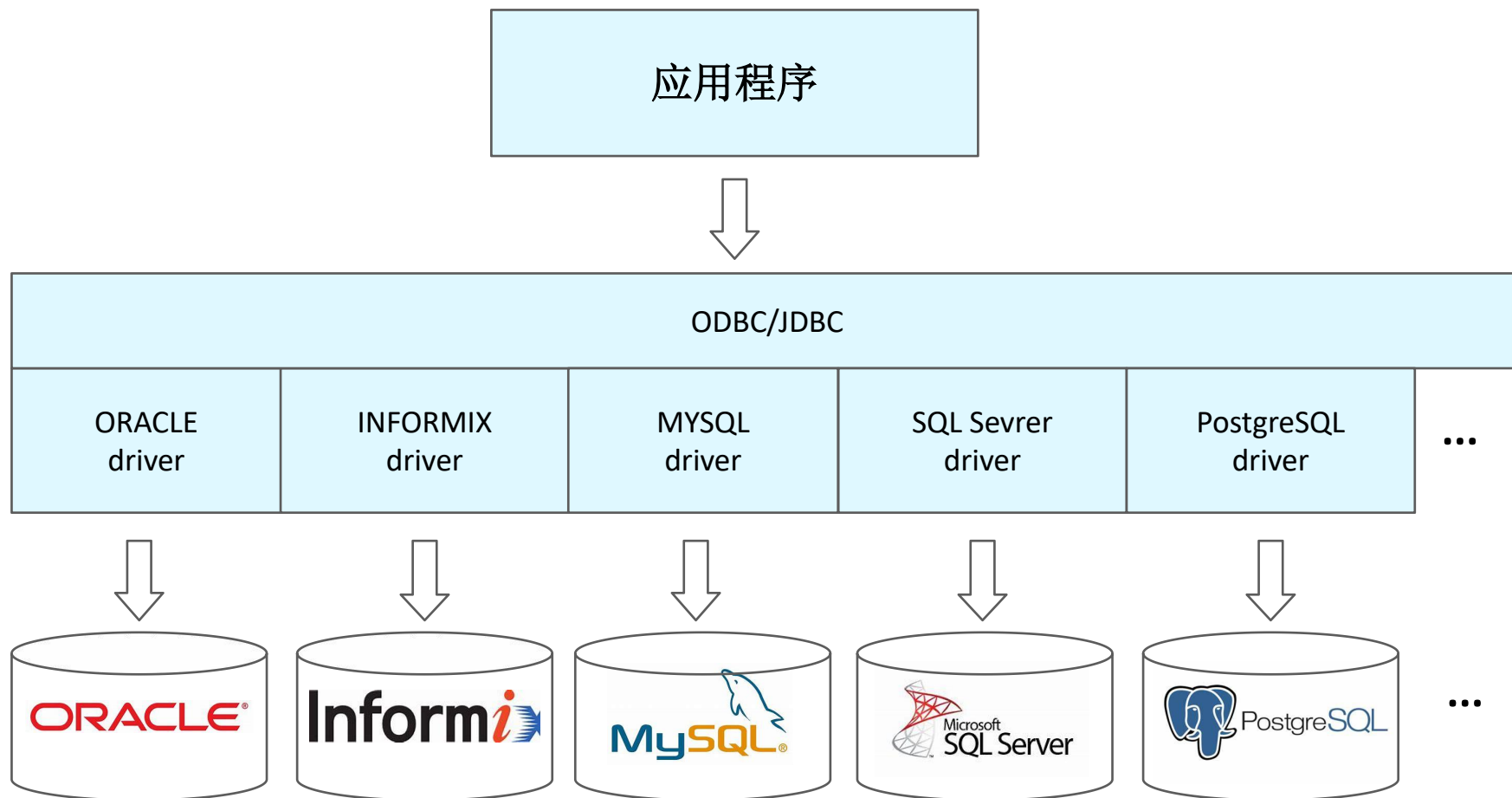
```
#include <arpa/inet.h>
int encodeIntLE(int i, char* p) // 将整数编码为LITTLE-ENDIAN格式存入p
{
    if(htonl(1) == 1){// 如果是BE系统，则htonl没有效果，该布尔表达式为真
        i = (((i) & 0xff000000u) >> 24)
            | (((i) & 0x00ff0000u) >> 8)
            | (((i) & 0x0000ff00u) << 8)
            | (((i) & 0x000000ffu) << 24);
    }
    memcpy(p, (char*)(&i), sizeof(int));
    return sizeof(int);
}
```

如果能够有其它手段判断，还是尽可能不要用宏定义。

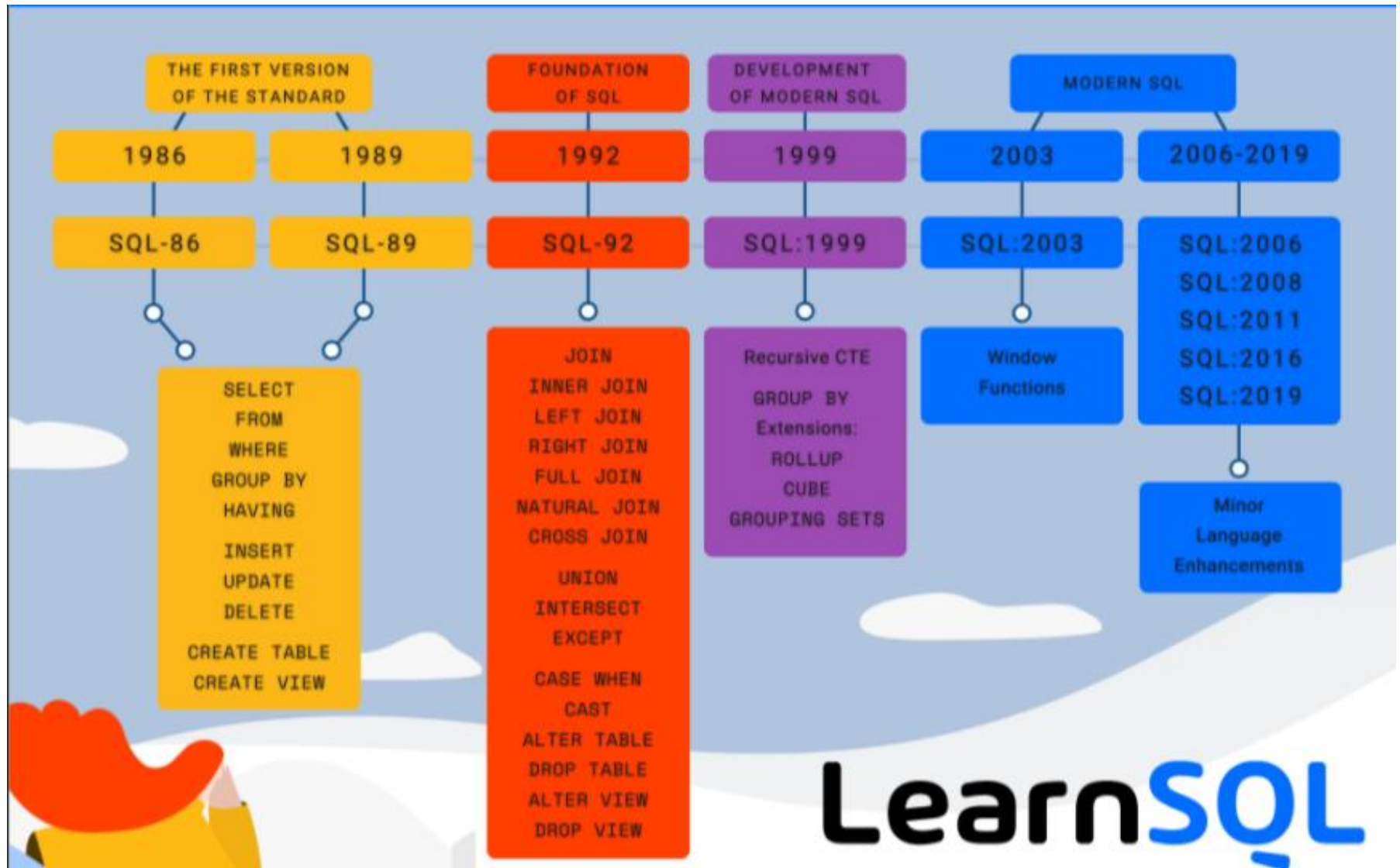
数据库和SQL - 早期的数据库访问方式



数据库和SQL - ODBC和JDBC



数据库和SQL - SQL的标准





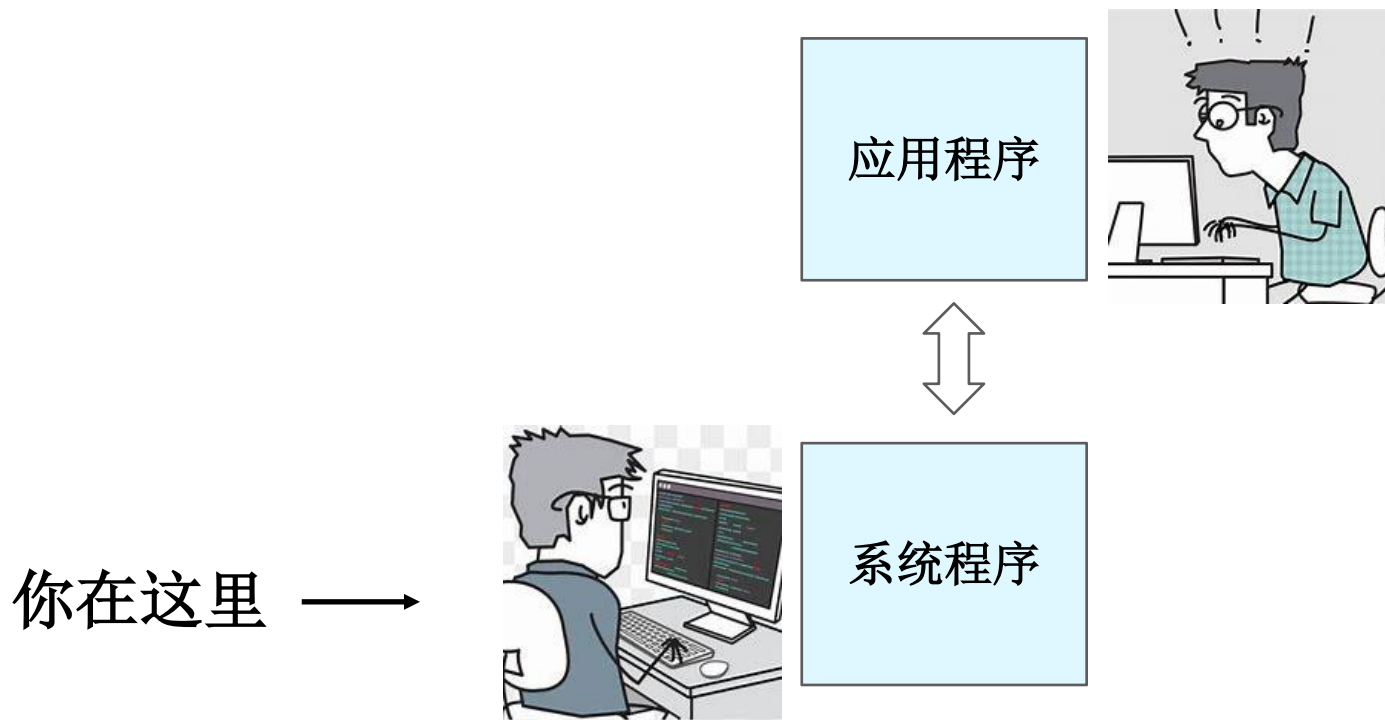
数据库和SQL - 不同数据库支持的SQL的差别

为了程序在数据库之间的可移植性，需要注意不同数据库之间的SQL的差别。举个例子，MYSQL和ORACLE在时间处理上的差别如下：

	ORACLE	MYSQL
字段类型	date	datetime
获取当前时间	<code>SYSDATE</code> <code>select SYSDATE from dual</code>	<code>now()</code> 或者 <code>SYSDATE()</code> <code>select now()</code>
时间转字符串	<code>to_char(hire_date,'yyyy" 年 "MM" 月 "DD" 日" HH:MI:SS AM')</code>	<code>DATE_FORMAT(value,'%Y 年%m 月%d 日')</code>
字符串转时间	<code>to_date('2021 年 11 月 10 日 11 点 30 分 ','yyyy"年"MM"月"DD"日"HH"点 "MI"分")</code>	<code>STR_TO_DATE('2021 年 11 月 10 日 ','%Y 年 %m 月 %d 日')</code>

很多应用程序为了解决这个问题，不使用date或者datetime，而是将时间字段定义为字符串类型，由应用进行格式处理。

换个角度看可移植性的问题



因为系统程序员的疏忽，系统程序的升级往往会给应用程序带来可移植性的问题。



echo的故事

```
$ echo hello world  
hello world
```

曾经，有一个“聪明”的系统程序员，对echo的功能做了这样的增强，可以解释\n了！

```
$ echo "hello\nworld"  
hello  
world
```

但是，导致之前版本上没问题的脚本，出现了问题

```
$ echo c:\home\nobel  
c:\home  
obel
```

现在的正确的echo程序是这样的：

```
$ echo -e "hello\nworld"  
hello  
world  
$ echo c:\home\nobel  
c:\home\nobel
```

如果命令的行为发生改变，那么应该另起一个名字，或者增加选项。

sum的故事

machine1上有个文件file, machine2上也有个文件file。如果我们想知道这两个文件是否相同, 可以分别求出两个文件的sum, 如果结果相等, 那么极大概率这两个文件相同。

```
machine1:~$ sum file  
52313 2 file
```

```
machine2:~$ sum file  
52313 2 file
```

曾经, 又有一个“聪明”的系统程序员, 对sum的做了改进, 改了更好的算法, 也许性能提高了, 也许hash效果更好了。但是, sum结果和以前的版本不一样了。这样会带来什么问题呢?

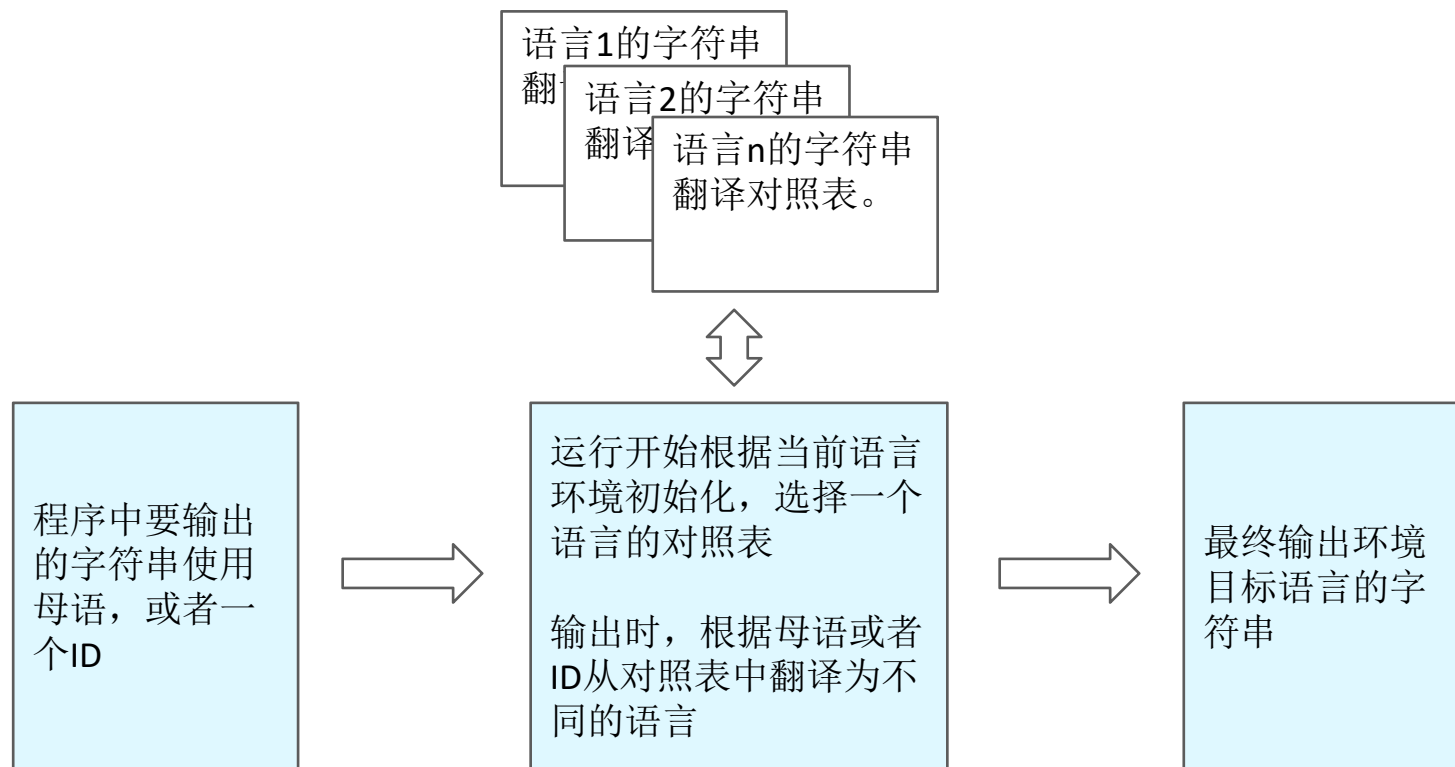
带来的问题是: **sum命令的这个重要作用完全丧失了。**

如果命令的行为发生改变
那么应该另起一个名字
或者增加选项

国际化(Internationalization)

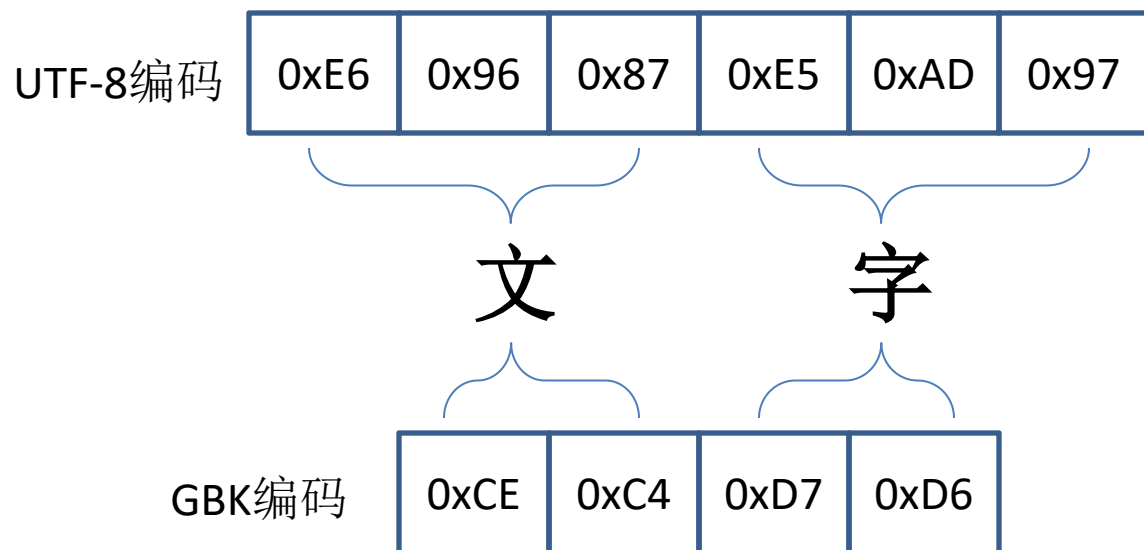
可移植性的另一方面，是要处理程序在跨越语言和文化边界时的可移植性问题。

国际化(Internationalization,或者i18n)，指的是设法使一个程序并不对其运行的文化环境有任何假定。主要涉及到用户界面上显示的文字的语言，格式，以及所用的字符集。一般流程如下：



字符编码

字符编码：将现有的语言文字符号用什么样的二进制来表示。
每个国家都试图让自己的官方语言文字能够用尽可能短的编码来表示。
所以，这个世界上存在过超过一千种字符编码方式...



```
printf("文字\n");
```

```
printf("\xE6\x96\x87\xE5\xAD\x97\n");
```

```
printf("\xCE\xC4\xD7\xD6\n");
```

UNICODE简介

为了解决全球的文字兼容性的问题，首先，将每一个有意义的字母，符号，文字都给出一个唯一的身份编号，这就是UNICODE

例如：

- 65-90：大写拉丁字母A-Z
- 97-122：小写拉丁字母a-z
- 904-974：希腊字母，例如 α (945)， β (946)
- 1040-1103：西里尔字母（俄语字母）
- 12353-12538：日语假名
- 19968-40959：中日韩统一表意文字，例如：一(19968)，文(25991)，字(23333)，𠂇(40917)
- 128512-128591：Emoji，例如：😄(128512)，🤖(128561)
- 131072-191471：中日韩文字扩展，例如：𠂇(131072)，𠂇(177986)



<https://home.unicode.org/>

UNICODE的实现

UNICODE只是所有符号的逻辑上的身份编号，要实现信息交互，包括：

- 程序将文字呈现给用户
- 用户将文字输入到程序
- 程序之间的文字传递

需要将这个**UNICODE**编号实现为一种具体的编码形式。

HTML

```
<p>&#25991;&#23383;</p>
```

浏览器

文字

C程序

```
printf("\u6587\u5B57");
```

可执行程序(utf-8)

E6 96 87 E5 AD 97

终端

文字

手机1发短信

文字

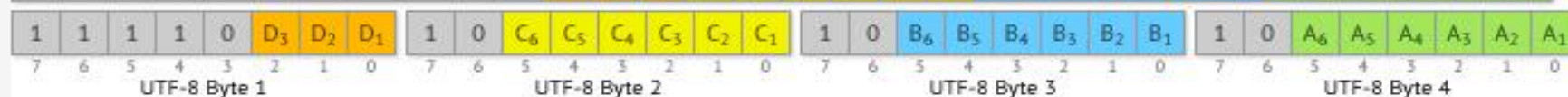
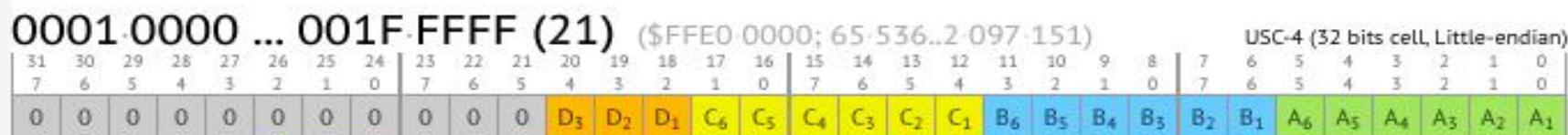
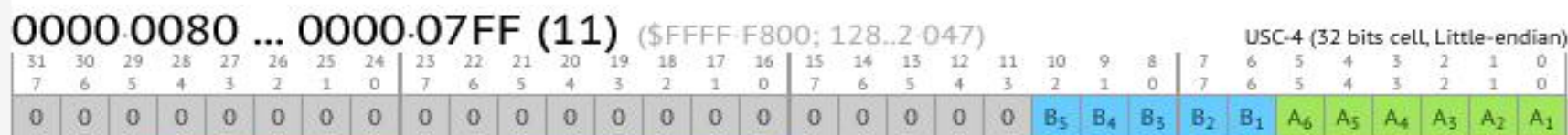
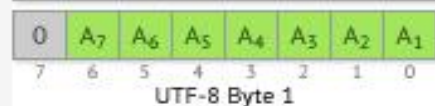
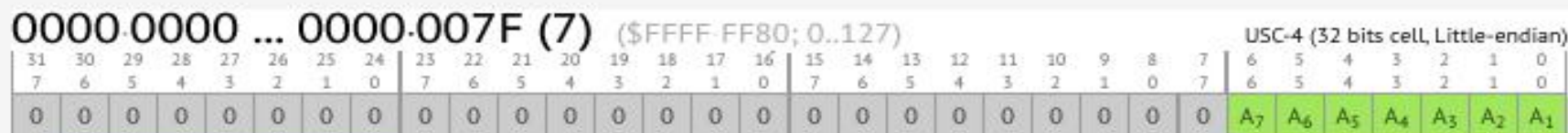
信令网络传输(ucs2)

65 87 5B 57

手机2收短信

文字

UTF-8简介



zh_CN.UTF-8



- 由于当前终端只能对UTF-8编码的文字正确解释，所以如果ls程序输出GBK编码的文字的时候，就会显示乱码。
需要修改终端配置解决。

国际化工具: gettext

hello_zh.po文件

```
msgid: "Hello World!"  
msgstr: "你好世界！"  
  
msgid: "..."  
msgstr: "..."
```

手工编写
或工具辅助生成

msgfmt
工具

hello.mo文件

hello.po的二进制形式，以便高效地读取加载。
不同的语言，生成不同的文件，位于不同的文件夹下。

不同的语言对应不同的翻译对照文件

根据当前语言环境
查找替换

C/C++程序

```
...  
printf("%s\n", gettext("Hello World!"));  
...
```

你好世界！

如何让程序的可移植性更好一些？

一开始就选择更高级的语言，例如Java在移植方面的问题就少一些。

不同环境下存在差异的语言特性，最好不要使用它。

谨慎启用语言的高版本特性，虽然这些特性用起来非常方便。

谨慎启用高版本的库，虽然高版本的库支持更多更好用的功能。

尽可能使用全球最通用的字符编码，例如UTF-8。

条件允许的情况下，将程序在尽可能多的环境下编译，测试，尽早发现程序中的一些移植问题。

【最重要的一条】做好设计，尽可能将那些系统依赖性高的代码集中在一起，隐藏在统一接口后面，让大部分代码和系统无关。

必要的地方，使用条件编译。条件编译是最后的选择，如果能用其它方法解决，就不要用条件编译。

隐藏系统依赖性的代码

